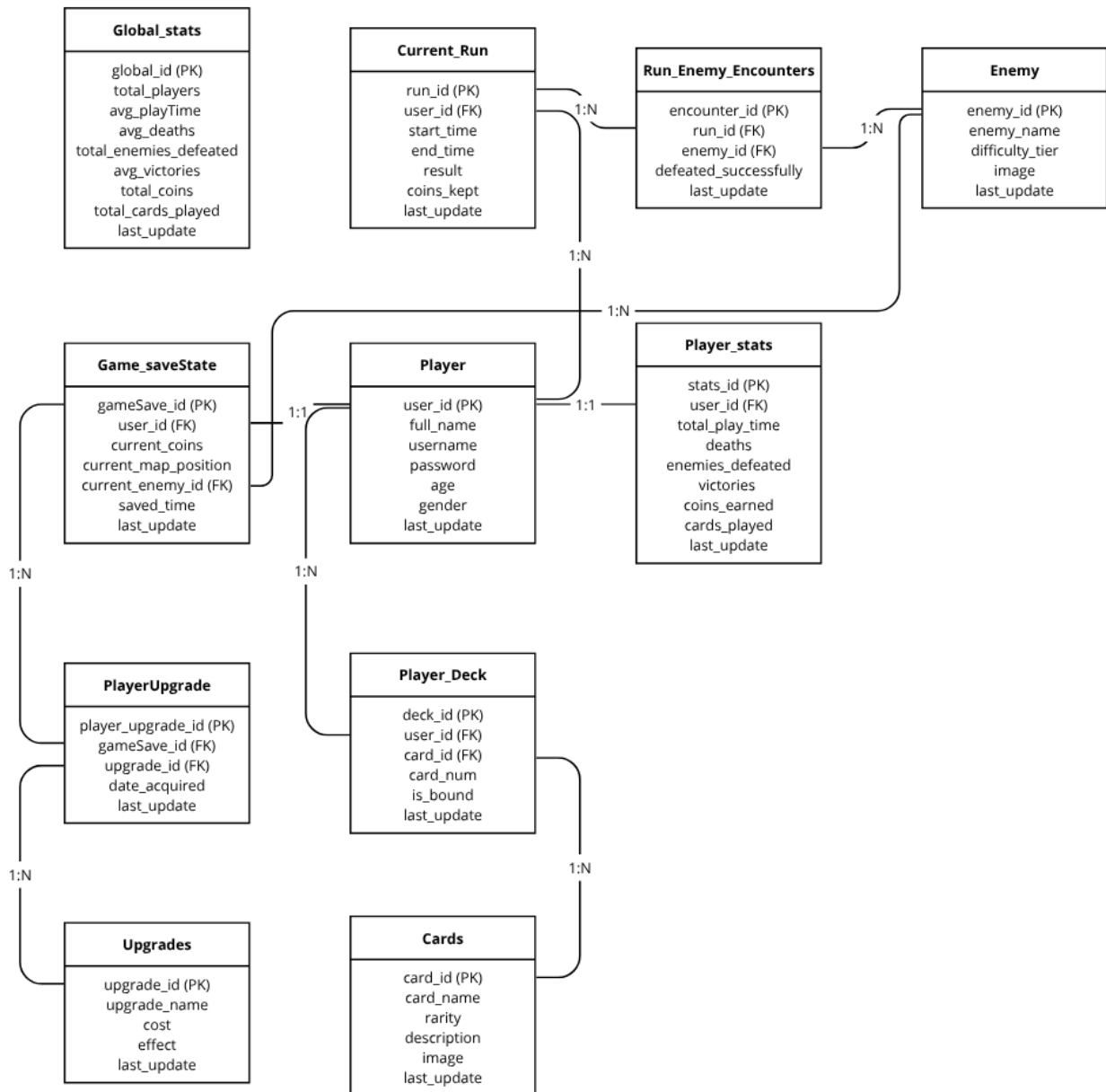


The Fool's Descent - Database Normalization for the Project Supported by Generative AI Resources

Regina Fernanda Portela Palacios

Miguel Eduardo Vega Bisognó

Santiago Aguilar Mello



Justification

Fixes based on the class concepts

- *The Run-to-Enemy Problem:* In the original design, connecting Current_Run directly to Enemy with a 1:N line meant a player could only fight one enemy per run, or an enemy could only ever exist in one run. To fix this classic 3NF transitive dependency, we built a junction table: Run_Enemy_Encounters. Now, a run can have many enemies, and an enemy can appear in many runs.
- *Flipped Cardinality Directions:* A few lines had their 1 and N tags backwards. For example, Player to Player_Deck showed that a deck has many players, instead of a player having many cards/decks. We flipped them so the Foreign Keys ($X \rightarrow Y$) actually line up with the Primary Keys.
- *Ditching Calculated Data:* Global_stats was tied directly to Current_Run. In a strict normalized database, you don't map live transactional rows to global averages. We unlinked it because global stats are calculated on the side, meaning they shouldn't have a direct hard-coded relationship line to a single run.

Fixes suggested by AI (Gemini)

- *Separating Live Gameplay from Analytics (OLTP Decoupling):* By untangling Global_stats from the live gameplay tables, we ensured that the server isn't trying to recalculate global world averages for thousands of players at the exact millisecond someone finishes a match. It keeps the game running fast.
- *Smart Pointers for Save States:* In Game_saveState, keeping current_enemy_id purely as a reference ID (a pointer) ensures we are just saving where the player left off. It stops the database from cloning the entire enemy asset folder inside a single player's save file.
- *Future-Proofing the Encounter Data:* By adding that new intermediate table for encounters, we can now track extra stats later on without needing to redesign the whole database structure.

How AI and Developers can actually work together

Using an AI tool to refine a database schema highlights exactly how the developer-AI relationship should look.

We need to think of AI as a highly specialized structural inspector. It is great at scanning a blueprint, instantly catching messy relationship lines, finding hidden 3NF violations and flagging potential performance bottlenecks before we write a single line of SQL.

However, AI cannot replace the developer. It doesn't know the exact pacing of the game, how difficult we want it to be or how the specific mechanics impact gameplay fun. AI provides the clean, mathematically sound foundation, but the developer is the architect who applies real-world judgment to make the final executive design choices.